Document number: P0299R0
Date: 2016-03-05
Author: Torvald Riegel <triegel@redhat.com>
Audience: LWG

P0299R0: Forward progress guarantees for the Parallelism TS v2

This paper contains wording that clarifies the forward progress requirements in the the Parallelism TS. See P0072R1 for background.

Wording

Apply the following changes to N4505. First, add a subsection titled ``Forward progress'' before Section 4.1.2, with the following content.

A thread of execution providing *parallel forward progress guarantees* is not guaranteed to make progress if it has not yet executed any execution step; once it has executed a step, the implementation will ensure that the thread will eventually make progress for as long as it has not terminated. [Note: The latter is required regardless of whether or not other threads of executions (if any) have been or are making progress. --- end note]

[Note: This effectively makes the same forward progress requirements as for a concurrent forward progress guarantee once the parallel-forward-progress thread of execution is started, but does not specify a requirement for when to start this thread of execution; the latter will typically be specified by the entity that creates this thread of execution. For example, executing tasks from a set of tasks in an arbitrary order, one after the other, satisfies the requirements of parallel forward progress for these tasks. --- end note]

A thread of execution providing *weakly parallel forward progress guarantees* is not guaranteed to make progress.

[Note: Threads of execution providing weakly parallel forward progress guarantees cannot be expected to make progress regardless of whether other threads make progress or not; however, blocking with forward progress guarantee delegation as defined below can be used to ensure that such threads of execution make progress eventually. --- end note]

[Note: Concurrent forward progress guarantees are stronger than parallel, which in turn are stronger than weakly parallel guarantees. For example, some kinds of synchronization between threads of execution may only make progress if the respective threads of execution provide parallel forward progress guarantees, but will fail to make progress under weakly parallel guarantees. --- end note]

If a thread of execution P is specified to *block with forward progress guarantee delegation* to block on the completion of a set S of threads of execution, and if P is providing a stronger forward progress guarantee than at least one thread of execution in S, then throughout the whole time of P being blocked on S, the implementation shall ensure that the forward progress guarantee provided by at least one thread of execution in S is at least as strong as P's forward progress guarantee. It is unspecified which thread of execution in S has its forward progress guarantee being potentially strengthened and for which number of execution steps. [Note: The strengthening is not permanent and not necessarily in place for the rest of the lifetime of the affected thread of execution. --- end note] As long as P is blocked, the implementation has to eventually select and potentially strengthen a thread of execution in S. Once a thread of execution in S terminates, it is removed from S. Once S is empty, P is unblocked.

[Note: A thread of execution B thus can temporarily provide an effectively stronger forward progress guarantee for a certain amount of time, due to a second thread of execution A being blocked on it with forward progress guarantee delegation. In turn, if B then blocks with forward progress guarantee delegation on C, this may also temporarily provide a stronger forward progress guarantee to C.--- end note]

[Note: If all threads of execution in S finish executing (e.g., they terminate and do not use blocking synchronization incorrectly), then P's execution of the operation that blocks with forward progress guarantee delegation will not result in P's progress guarantee being effectively weakened. --- end note]

[Note: This does not remove any constraints regarding blocking synchronization for threads of execution providing parallel or weakly parallel forward progress guarantees because the implementation is not required to strengthen a particular thread of execution whose too-weak progress guarantee is preventing overall progress. --- end note]

Adapt the former 4.1.2 as follows (which is now 4.1.3 after the previous change).

Generally, to avoid misinterpretations, replace all occurrences of ``thread'' with ``thread of execution'' (see N4231 for background); given that this is a mechanical change, these changes are not spelled out.

Next, specify which forward progress guarantees are provided by threads of execution supplied by the library by changing 4.1.3p3 and 4.1.3p4:

> The invocations of element access functions in parallel algorithms invoked with an execution policy object of type `parallel_execution_policy` are permitted to execute in an unordered fashion in either the invoking thread **of execution** or in a thread **of execution** implicitly created by the library to support parallel algorithm execution**. If the threads of execution created by `std::thread` provide concurrent forward progress guarantees, then a thread of execution implicitly created by the library will provide parallel forward progress guarantees; otherwise, the provided forward progress guarantee is implementation-defined**. Any such invocations executing in the same thread of execution are indeterminately sequenced with respect to each other. [Note: It is the caller's responsibility to ensure correctness, for example that the invocation does not introduce data races or deadlocks. --- end note]
>
> *[No change to the examples]*
>
> The invocations of element access functions in parallel algorithms invoked with an execution policy of type `parallel_vector_execution_policy` are permitted to execute in an unordered fashion in unspecified threads **of execution**, and unsequenced with respect to one another within each thread **of execution. These threads of execution are either the invoking thread of execution or threads of execution implicitly created by the library; the latter will provide weakly parallel forward progress guarantees.** [Note: This means that multiple function object invocations may be interleaved on a single thread of execution. --- end note]
>
> [Note: This overrides the usual guarantee from the C++ standard, Section 1.9 [intro.execution] that function executions do not interleave with one another. --- end note]
>
> Since `parallel_vector_execution_policy` allows the execution of element access functions to be interleaved on a single thread~~,~~ **of execution, blocking** synchronization, including the use of mutexes, risks deadlock. Thus**,** the synchronization with `parallel_vector_execution_policy` is restricted as follows:
>
> A standard library function is *vectorization-unsafe* if it is specified to synchronize with another function invocation, or another function invocation is specified to synchronize with it, and if it is not a memory allocation or deallocation function. Vectorization-unsafe standard library functions may not be invoked by user code called from `parallel_vector_execution_policy` algorithms.
>
> [Note: Implementations must ensure that internal synchronization inside standard library routines does not ~~induce deadlock~~**prevent forward progress when those routines are executed by threads of execution with weakly parallel forward progress guarantees**. This can be achieved by either avoiding synchronization incompatible with those guarantees, or by executing these routines differently. --- end note]

Insert a new paragraph before paragraph 6:

> **If an invocation of a parallel algorithm uses threads of execution implicitly created by the library, then the invoking thread of execution will either block with forward progress guarantee delegation on the completion of these library-managed threads of execution or will eventually execute element access functions. [Note: In blocking with forward progress guarantee delegation in this context, a thread of execution created by the library is considered to have finished execution as soon as it has finished the execution of the particular element access function that the invoking thread of execution logically depends on.]**